



Universidad
Zaragoza



Sistemas Inteligentes I

Lección 5.

Juegos (2da Parte)



Índice

- Juegos
- Decisiones optimas
- Poda α - β
- Juegos con información imperfecta
- Juegos de azar

Juegos

- Los juegos son una forma de entorno *multi-agente*
 - ¿Qué hacen los otros agentes y como afectan sus acciones a nuestro éxito?
 - Los entornos multiagentes competitivos dan lugar a problemas de búsqueda con adversario (juegos)
- ¿Por qué tratar los juegos?
 - divertidos; entretenidos
 - Materia de estudio interesante por que son problemas difíciles
 - P.E. Ajedrez, el factor de ramificación es 35 de media, y los juegos suelen dar lugar a 50 movimientos por cada jugador, da un árbol de búsqueda de $35^{100} = 10^{154}$
 - Fácil de representar como problemas y con agentes que presentan un restringido número de acciones

Relación de los juegos con la búsqueda

- Búsqueda– no hay adversario
 - La solución es un método (heurística) para encontrar un objetivo
 - Las heurísticas y las técnicas CSP (constraint satisfaction problem) pueden encontrar una solución óptima.
 - Función de evaluación: estima el coste desde el inicio al objetivo a través de un nodo dado.
 - Ejemplos: búsqueda de caminos, planificación de actividades
- Juegos– con adversario
 - La solución es una estrategia (la estrategia especifica el movimiento a cada replica del oponente)
 - La limitación en tiempo fuerza a una solución aproximada
 - La función de evaluación: evalúa lo buena que es una posición del juego
 - Ejemplos: ajedrez, damas, Othello, backgammon

Tipos de juegos

	Determinista	Azar
Información perfecta	Ajedrez, damas, othello	backgamon, monopoly
Información imperfecta		Bridge, poker, scrabble



Juegos típicos en IA

- Dos Jugadores: MAX y MIN
 - MAX mueve primero y alterna turnos con MIN hasta que el juego acaba. El ganador recibe el premio/beneficio, el perdedor es penalizado.
 - Entornos con más de dos agentes se ven como economías en lugar de juegos.
- Juegos suma-cero de información perfecta (ajedrez)
 - Determinista, completamente observables, los valores de utilidad al final del juego son siempre opuestos, y su suma es siempre igual.

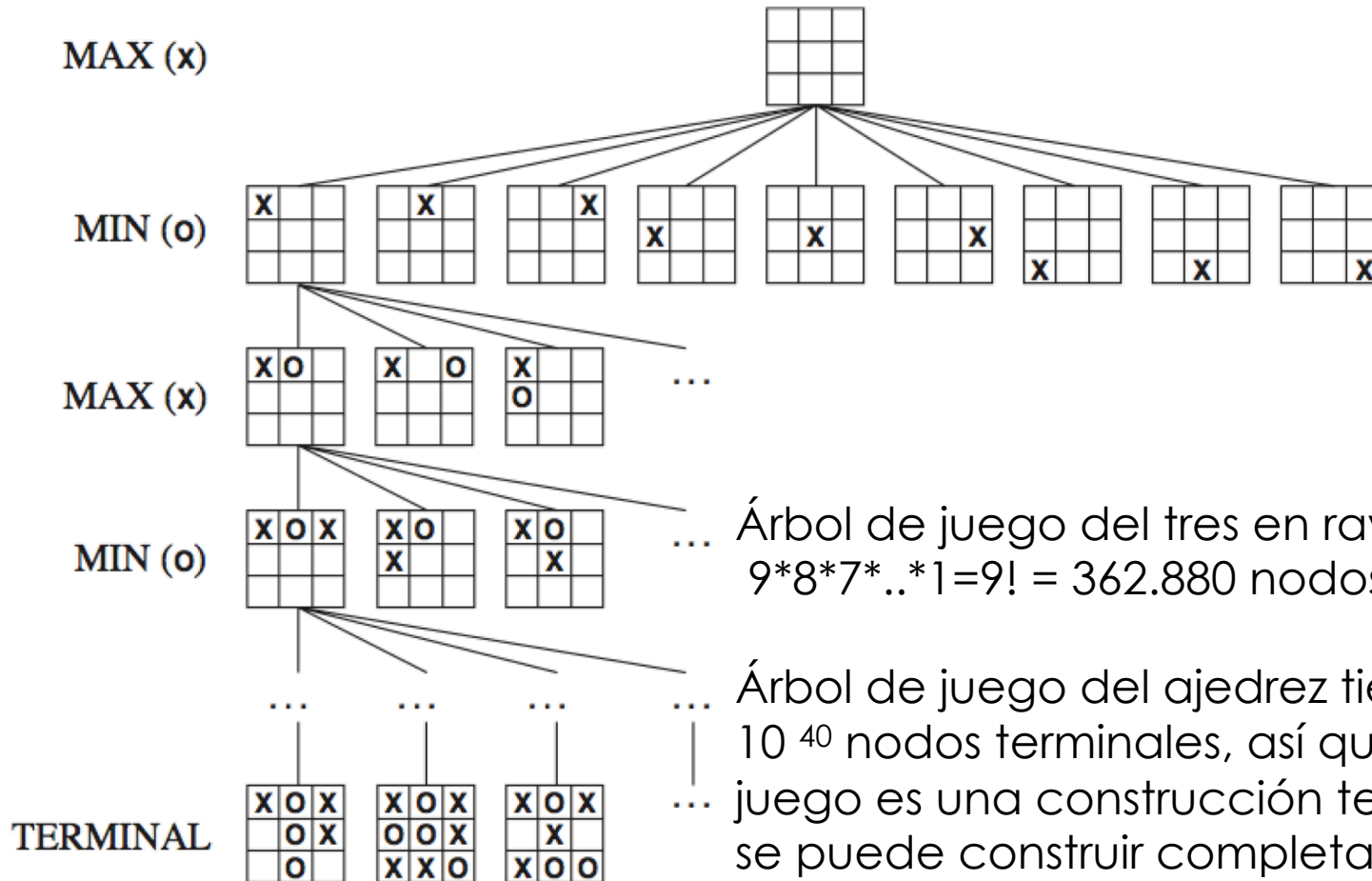
Juegos como búsqueda

- **S_0 , Estado inicial:** p.e. Tablero inicial ajedrez
- **Jugador(s):** Define jugador que mueve en un estado
- **Acciones(s):** Movimientos legales en un estado
- **Resultado(s,a): Modelo de transición,** que define el resultado de un movimiento.
- **Test-terminal(s): Estado terminal** si el juego está finalizado
- **Función Utilidad:** Valor numérico del estado terminal. P.e. gana(+1), pierde(-1) y empate(0) en juego tres en raya (a continuación)

Juegos como búsqueda

- MAX utiliza un **árbol de juego** para determinar el siguiente movimiento.
 - Nodos que representan estados del juego, y arcos que representan movimientos legales
- **Podar** (Pruning) el árbol nos permite ignorar partes del árbol de búsqueda
- **Funciones de evaluación heurística** nos permite aproximar la utilidad real de un estado sin hacer una búsqueda completa.

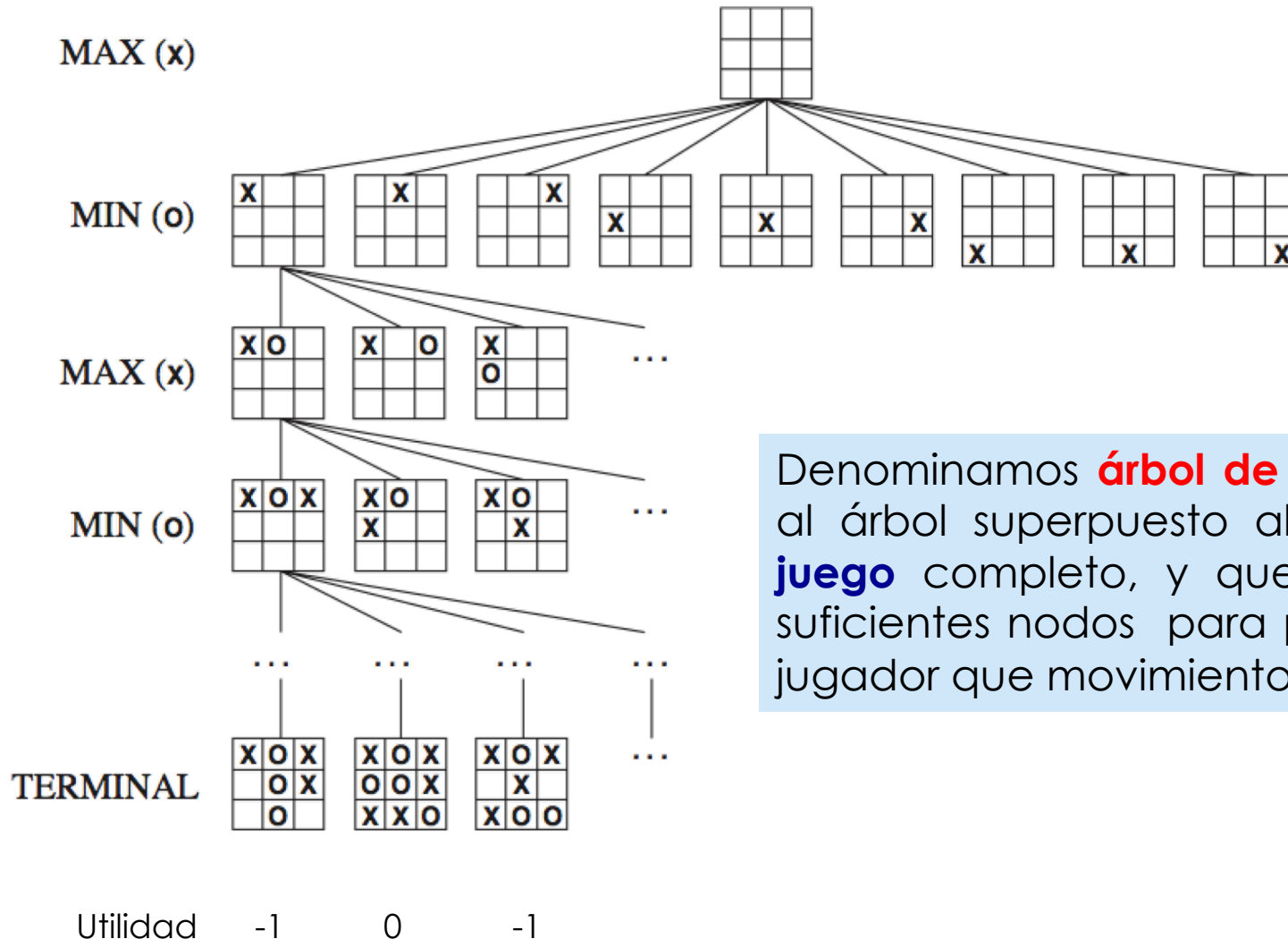
Árbol de juego del tres en raya



Árbol de juego del tres en raya tiene $9 \cdot 8 \cdot 7 \cdot \dots \cdot 1 = 9! = 362.880$ nodos terminales

Árbol de juego del ajedrez tiene 10^{40} nodos terminales, así que el árbol de juego es una construcción teórica que no se puede construir completamente

Árbol de juego del tres en raya



Denominamos **árbol de búsqueda** al árbol superpuesto al **árbol de juego** completo, y que examina suficientes nodos para permitir al jugador que movimiento realizar

Estrategias optimas

- Encuentra la estrategia de contingencia para MAX asumiendo que MIN es un oponente infalible.
- Suponemos que ambos jugadores juegan de forma óptima
- Dado un árbol de juego, la estrategia óptima puede ser determinado por el valor minimax de cada nodo:

VALOR-MINIMAX(s)=

UTILIDAD(s)

Si Test-terminal (s)

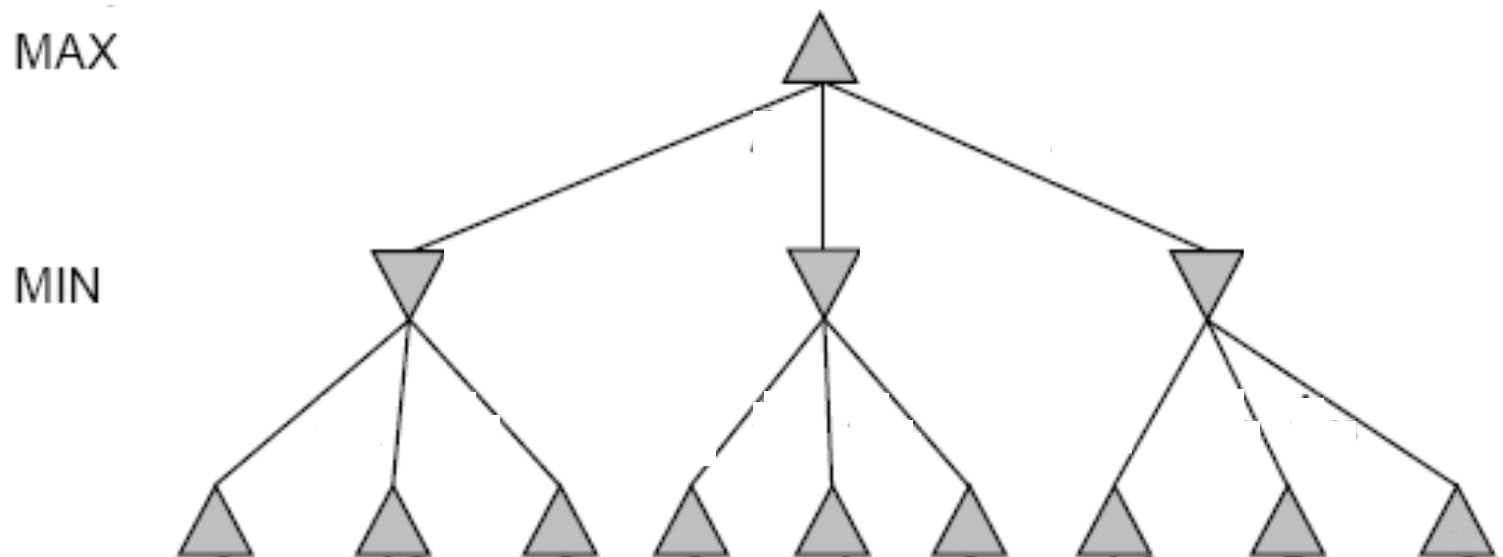
$\max_{a \in \text{Acciones}(s)} \text{MINIMAX}(\text{Resultado}(s,a))$

Si $\text{Jugado}(s)=\text{MAX}$

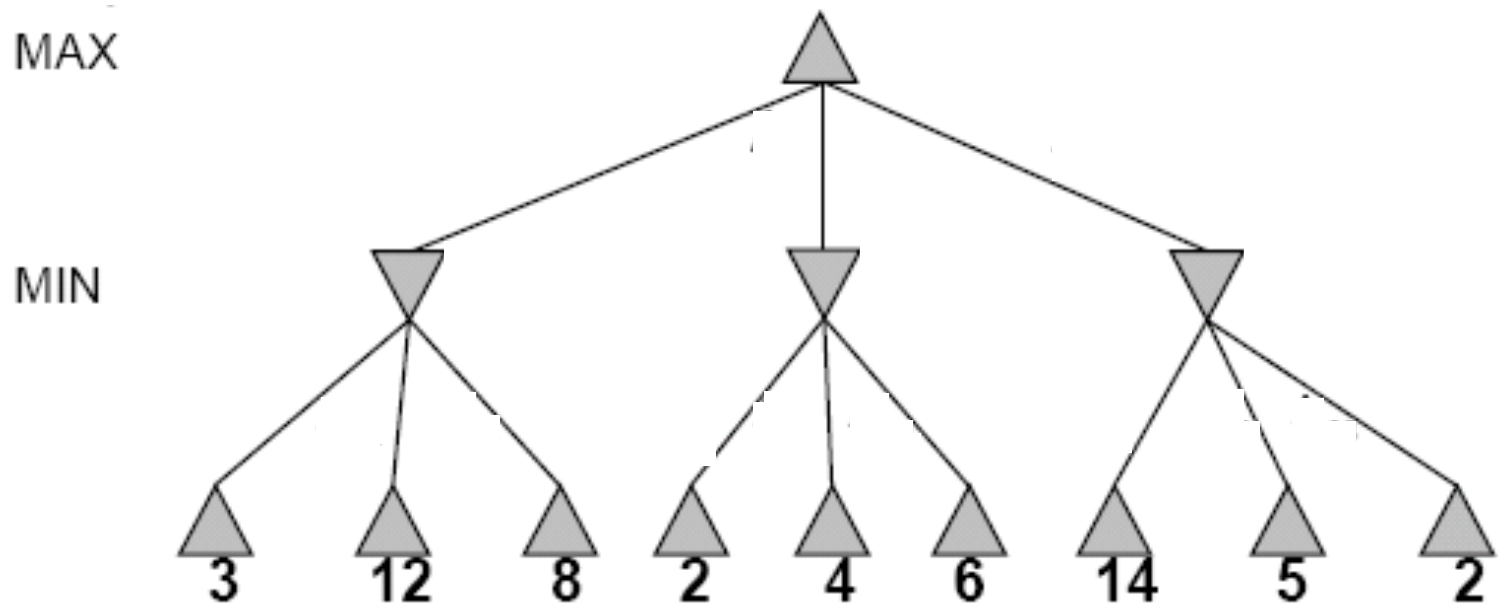
$\min_{a \in \text{Acciones}(s)} \text{MINIMAX}(\text{Resultado}(s,a))$

Si $\text{Jugador}(s)=\text{MIN}$

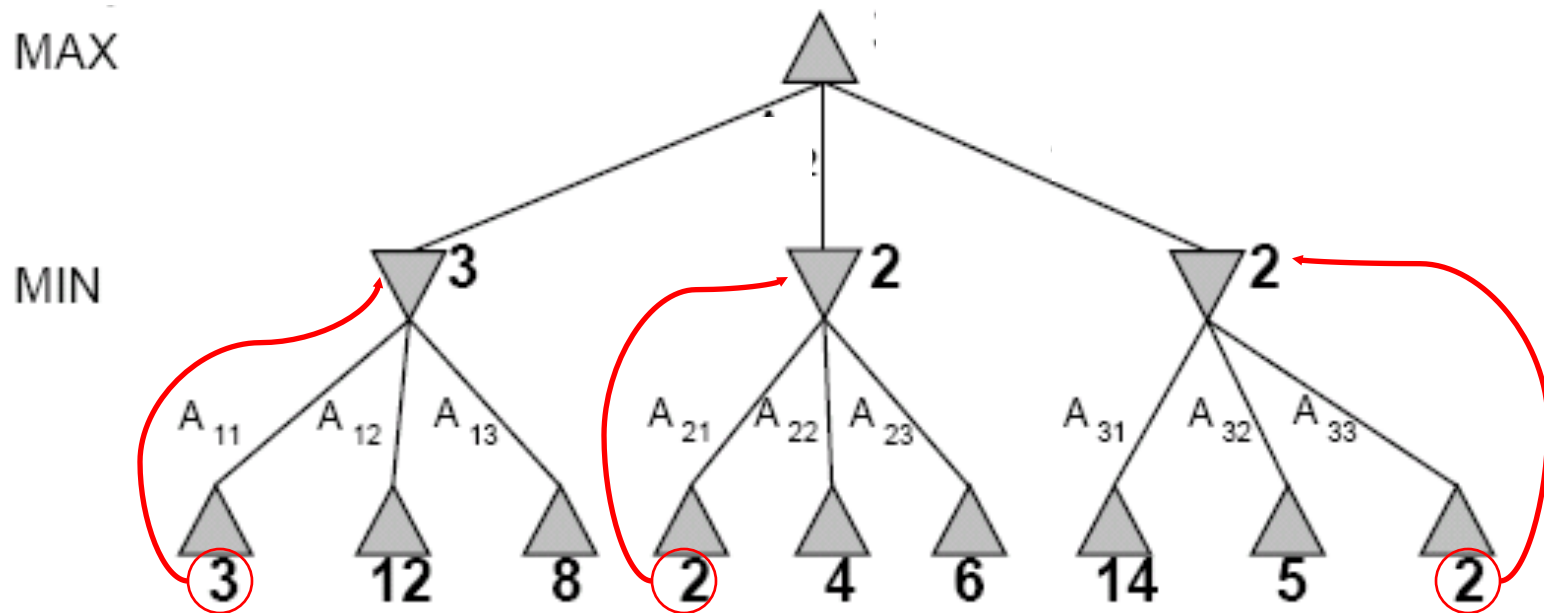
Árbol juego Dos-turnos



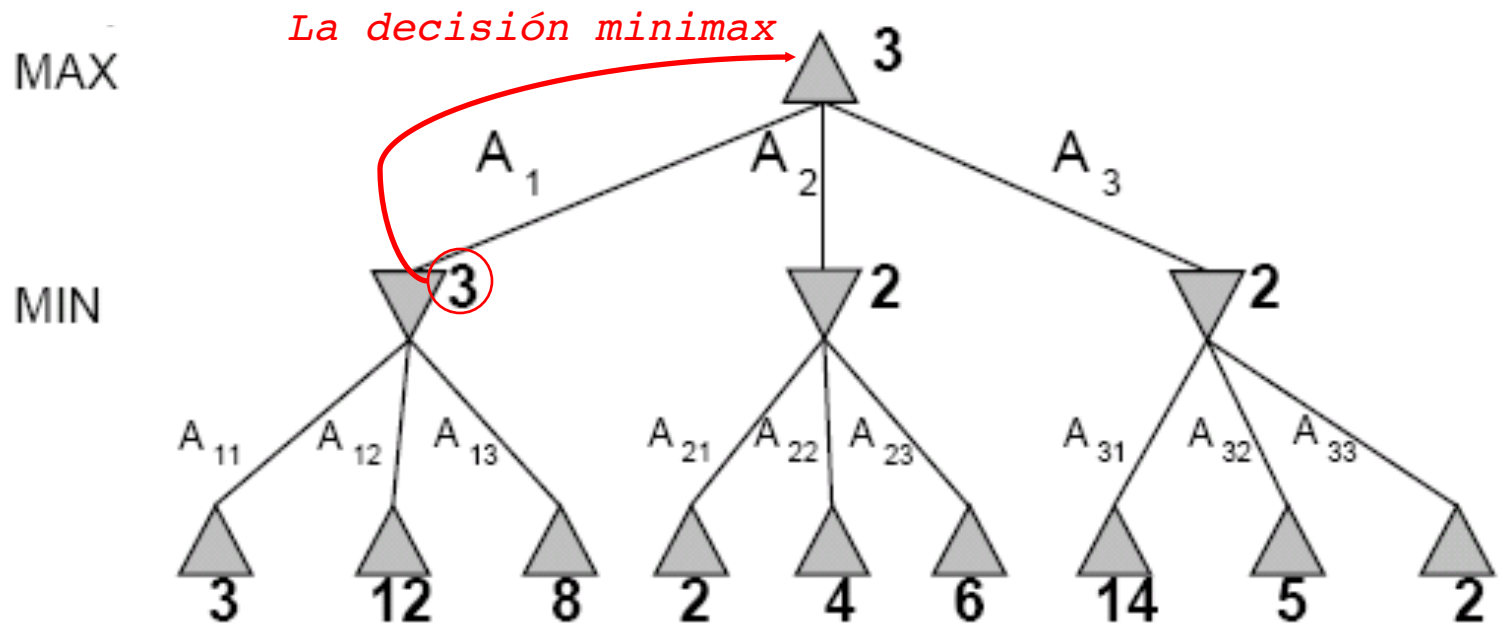
Árbol juego Dos-turnos



Árbol juego Dos-turnos



Árbol juego Dos-turnos



Minimax maximiza la utilidad/beneficio en el peor caso para max

¿Qué ocurre si MIN no juega de forma óptima?

- La definición de juego óptimo para MAX asume que MIN juega de forma óptima: Maximiza el máximo beneficio en el peor caso para MAX..
- Pero si MIN no juega optimamente, MAX lo hará todavía mejor.

Algoritmo Minimax

function MINIMAX-DECISION(*estado*) **returns** *una acción*

$v \leftarrow \text{MAX-VALOR}(\textit{estado})$

return la acción en Resultados(*estado*, *acción*) con valor v

function MAX-VALOR(*estado*) **returns** *un valor utilidad*

if TEST-TERMINAL (*estado*) **then return** UTILIDAD(*estado*)

$v \leftarrow -\infty$

for each a in Acciones(*estado*) **do**

$v \leftarrow \text{MAX}(v, \text{MIN-VALOR}(\text{Resultado}(s)))$

return v

function MIN-VALOR(*estado*) **returns** *un valor utilidad*

if TEST-TERMINAL (*estado*) **then return** UTILIDAD(*estado*)

$v \leftarrow \infty$

for each a in Acciones(*estado*) **do**

$v \leftarrow \text{MIN}(v, \text{MAX-VALOR}(\text{Resultado}(s)))$

return v



Propiedades Minimax

El algoritmo Minimax realiza una exploración completa primero en profundidad de un árbol de juego

m = Profundidad máxima del árbol
b = mov. legales en cada punto

Criterio	Minimax
¿Completo?	
¿Óptimo?	
Temporal	
Espacial	

Propiedades Minimax

El algoritmo minimax realiza una exploración completa primero en profundidad de un árbol de juego

m = Profundidad máxima del árbol
b = mov. legales en cada punto

¿Completo?

Sólo si el árbol es finito. Pero es posible una estrategia aunque no exista un árbol finito.

Criterio	Minimax
¿Completo?	Si 😊
¿Óptimo?	
Temporal	
Espacial	

Propiedades Minimax

El algoritmo minimax realiza una exploración completa primero en profundidad de un árbol de juego

m = Profundidad máxima del árbol
b = mov. legales en cada punto

¿Óptimo?

Si contra un oponente óptimo

Criterio	Minimax
¿Completo?	Si 😊
Óptimo	Si 😊
Espacial	
¿Óptimo?	

Propiedades Minimax

El algoritmo minimax realiza una exploración completa primero en profundidad de un árbol de juego

m = Profundidad máxima del árbol
b = mov. legales en cada punto

¿Óptimo?

Si contra un oponente óptimo

Criterio	Minimax
¿Completo?	Si 😊
Óptimo	Si 😊
Espacial	
¿Óptimo?	

Propiedades Minimax

El algoritmo minimax realiza una exploración completa primero en profundidad de un árbol de juego

m = Profundidad máxima del árbol
b = mov. legales en cada punto

Complejidad Temporal

$$O(b^m)$$



Para juegos reales, la complejidad temporal no es práctica, pero el algoritmo sirve como base de análisis de algoritmos más prácticos.

Criterio	Minimax
¿Completo?	Si 😊
¿Óptimo?	Si 😊
Temporal	$O(b^m)$ 😞
Espacial	

Propiedades Minimax

El algoritmo minimax realiza una exploración completa **primero en profundidad** de un árbol de juego

m = Profundidad máxima del árbol
b = mov. legales en cada punto

Complejidad Espacial

$O(bm)$



Para juegos reales, la complejidad temporal no es práctica, pero el algoritmo sirve como base de análisis de algoritmos más prácticos.

Criterio	Minimax
¿Completo?	Si ☺
¿Óptimo?	Si ☺
Temporal	$O(b^m)$ ☹
Espacial	$O(bm)$ ☺

Decisión óptima en juegos con mas de dos jugadores

- Los valores minimax se convierten en vectores
- El nodo devuelto al nodo n es siempre el vector del estado sucesor con el valor más alto del jugador eligiendo en n .

to move

A

$(1, 2, 6)$

B

$(1, 2, 6)$

$(-1, 5, 2)$

C

$(1, 2, 6)$

X

$(6, 1, 2)$

$(-1, 5, 2)$

$(5, 4, 5)$

A

$(1, 2, 6)$

$(4, 2, 3)$

$(6, 1, 2)$

$(7, 4, -1)$

$(5, -1, -1)$

$(-1, 5, 2)$

$(7, 7, -1)$

$(5, 4, 5)$

$(1, 2, 6)$

$(4, 2, 3)$

$(6, 1, 2)$

$(7, 4, -1)$

$(5, -1, -1)$

$(-1, 5, 2)$

$(7, 7, -1)$

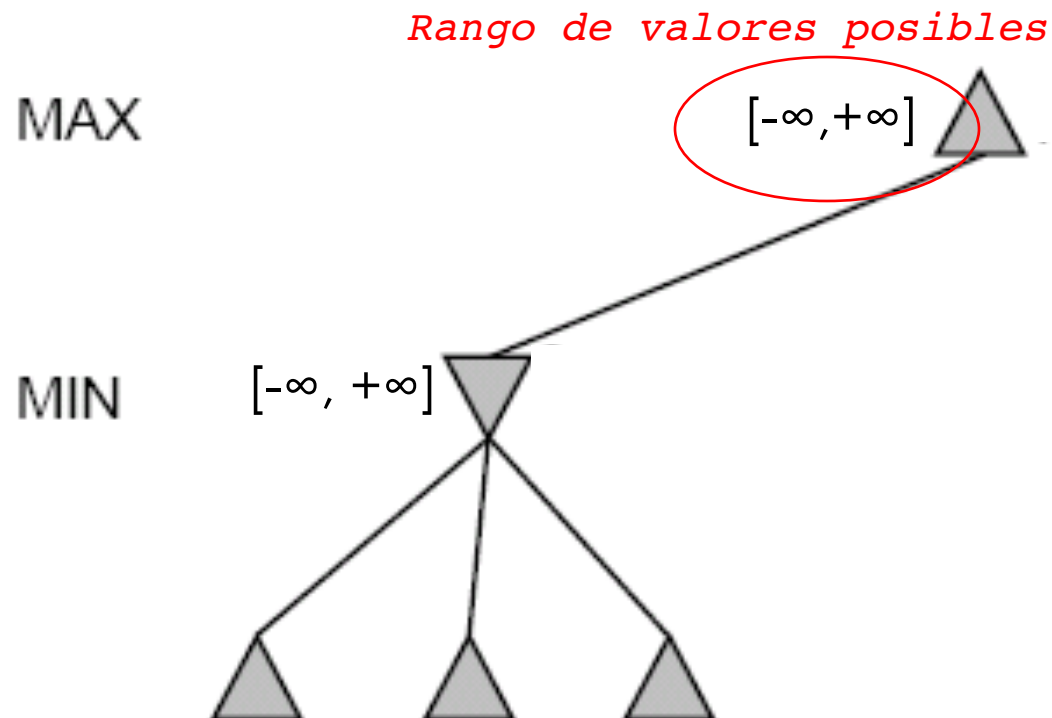
$(5, 4, 5)$

Problema de la búsqueda minimax

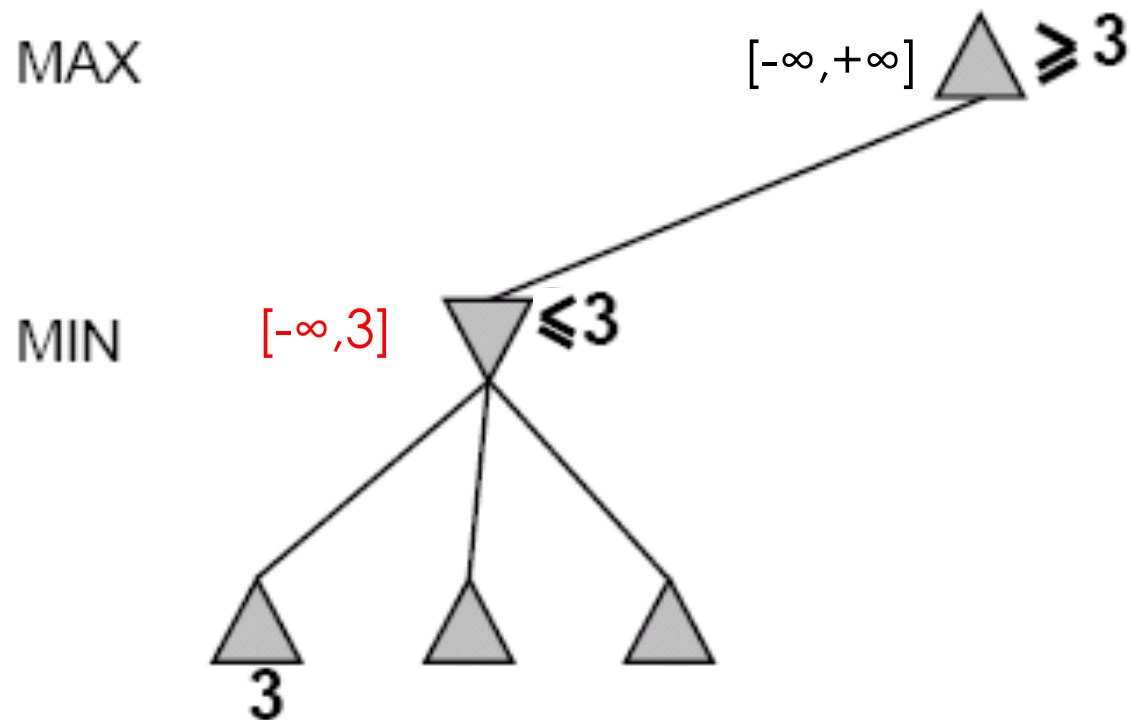
- El número de estados del juego es exponencial con el número de movimientos.
- **Solución:** No examinar cada nodo
- ==> **poda Alfa-beta**/Alpha-beta pruning
 - **Alfa**= valor de la mejor elección encontrada en cualquier punto del camino **MAX**
 - **Beta** = valor de la mejor elección encontrada en cualquier punto del camino **MIN**

Ejemplo Alpha-Beta

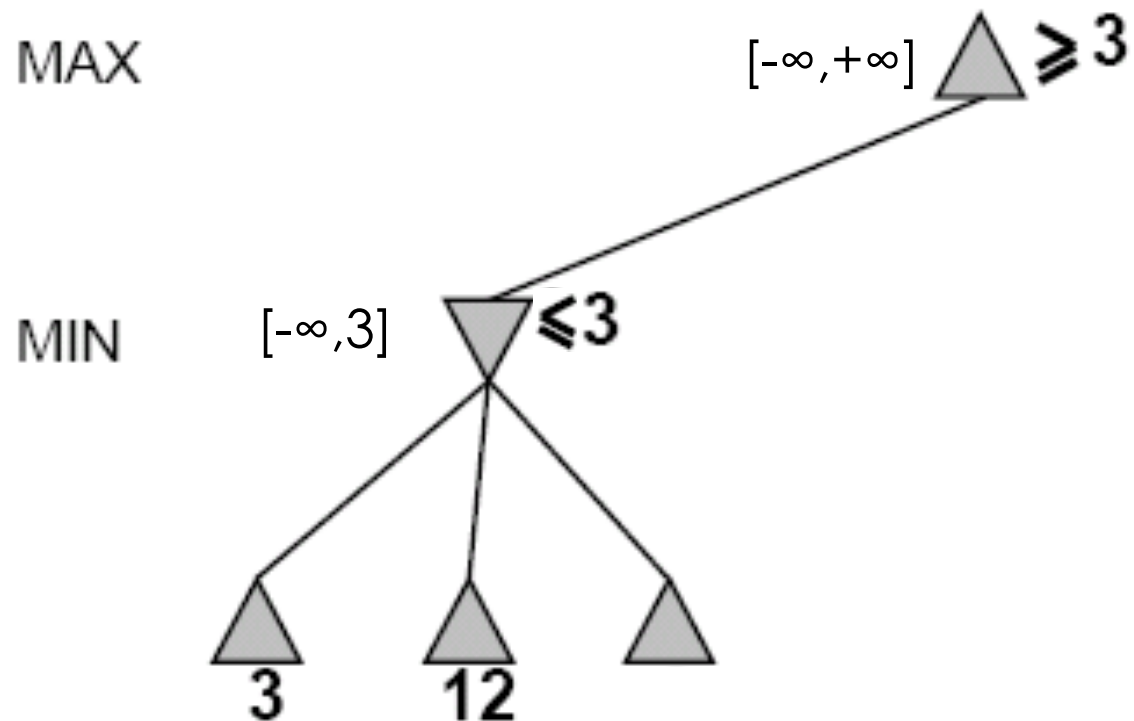
Hacer búsqueda primero en profundidad hasta un nodo hoja



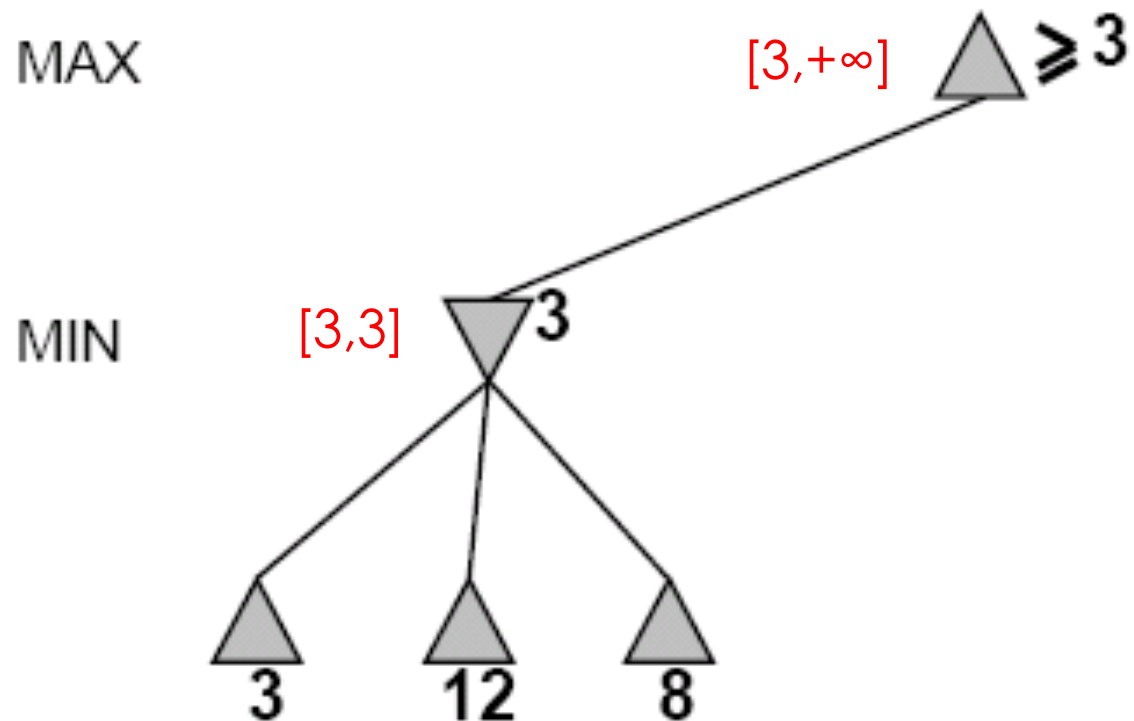
Ejemplo Alpha-Beta (continúa)



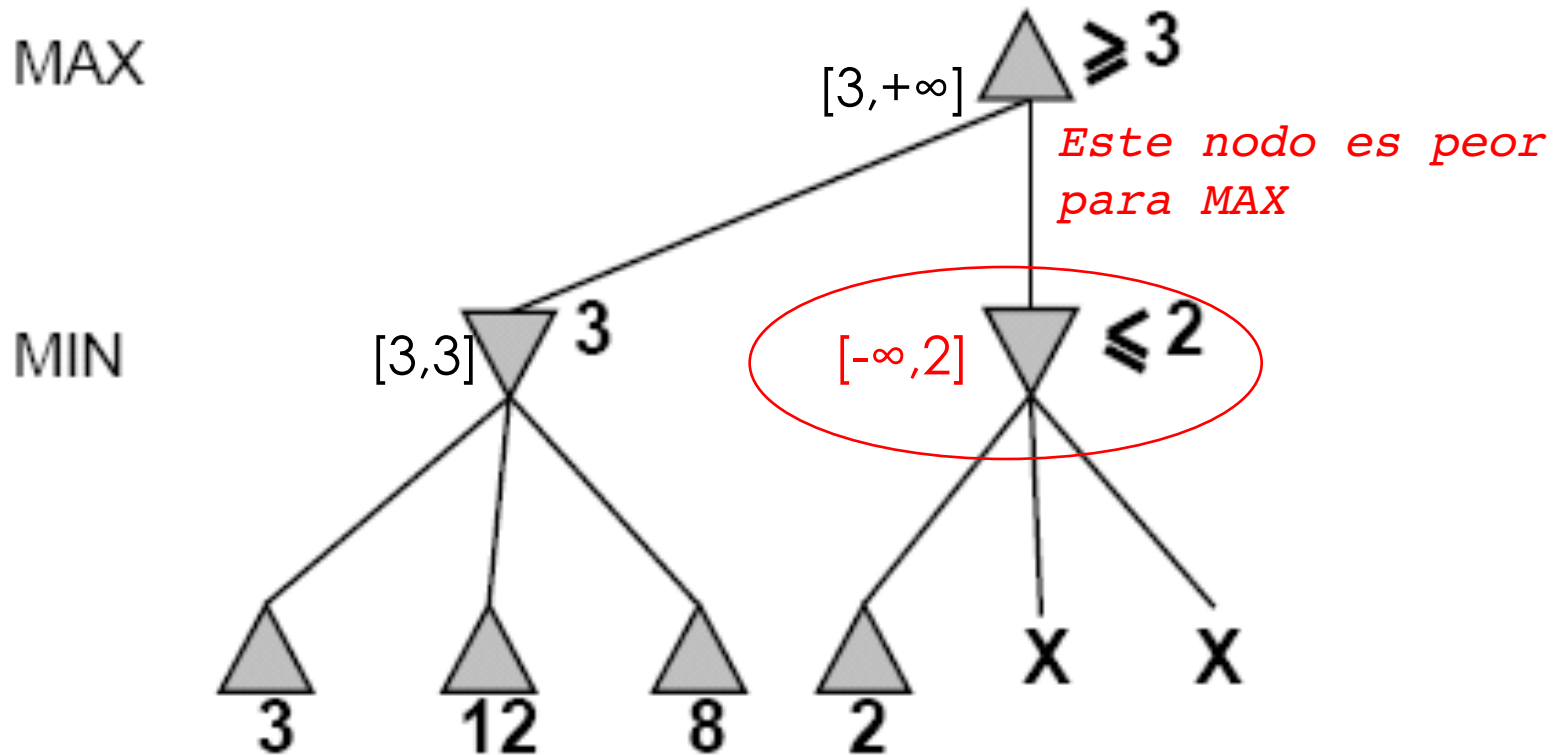
Ejemplo Alpha-Beta (continúa)



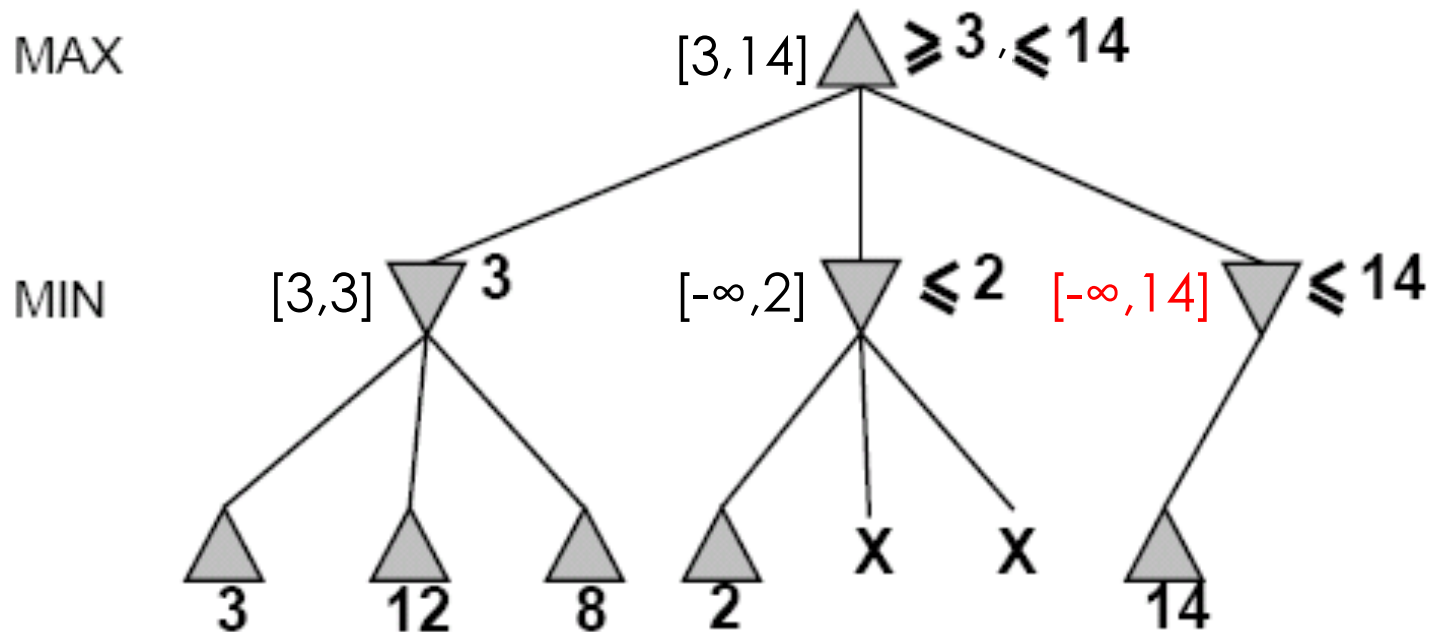
Ejemplo Alpha-Beta (continúa)



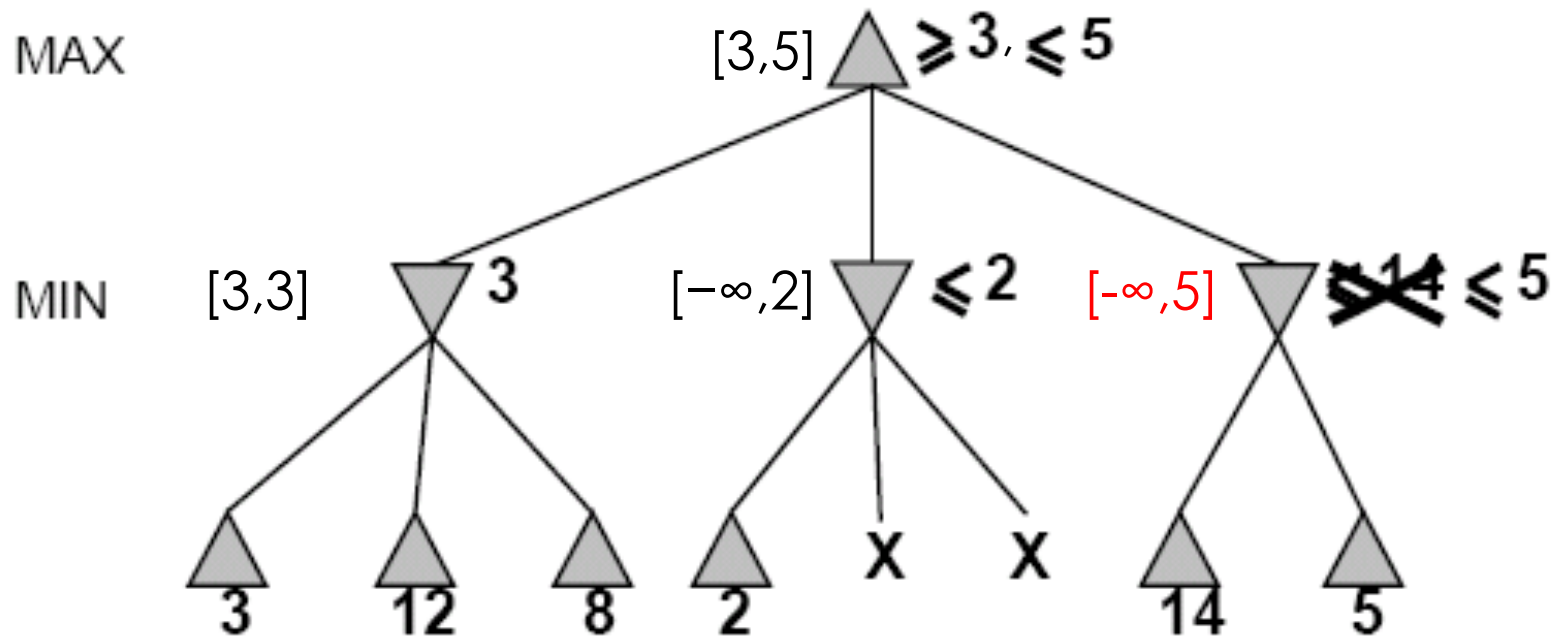
Ejemplo Alpha-Beta (continúa)



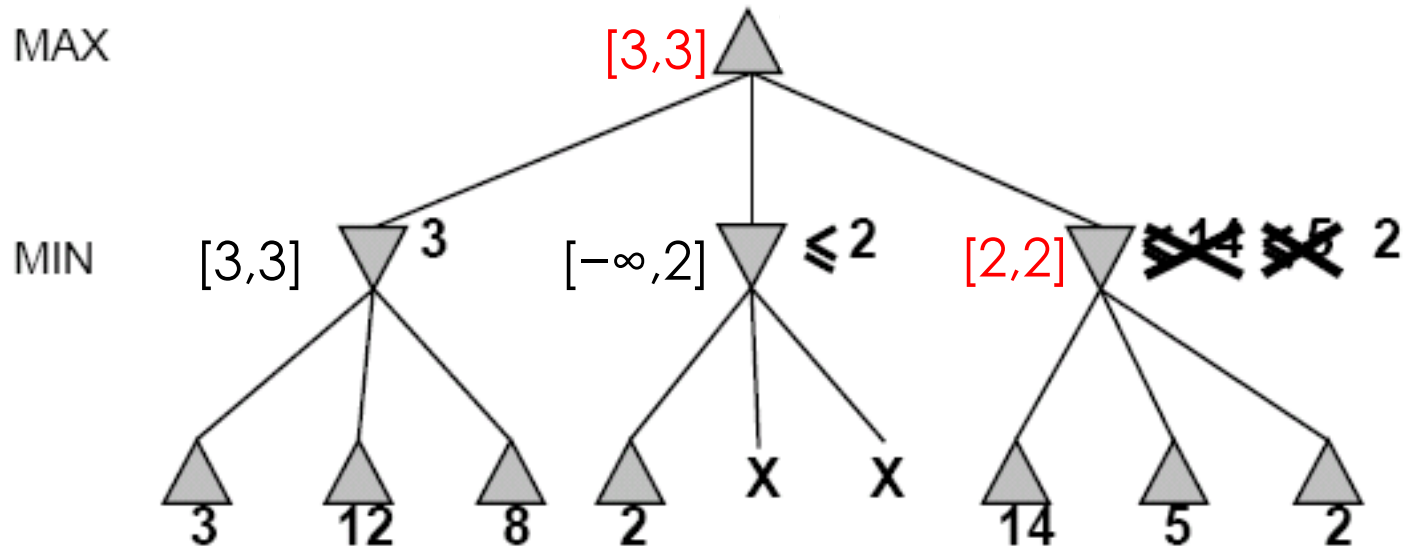
Ejemplo Alpha-Beta (continúa)



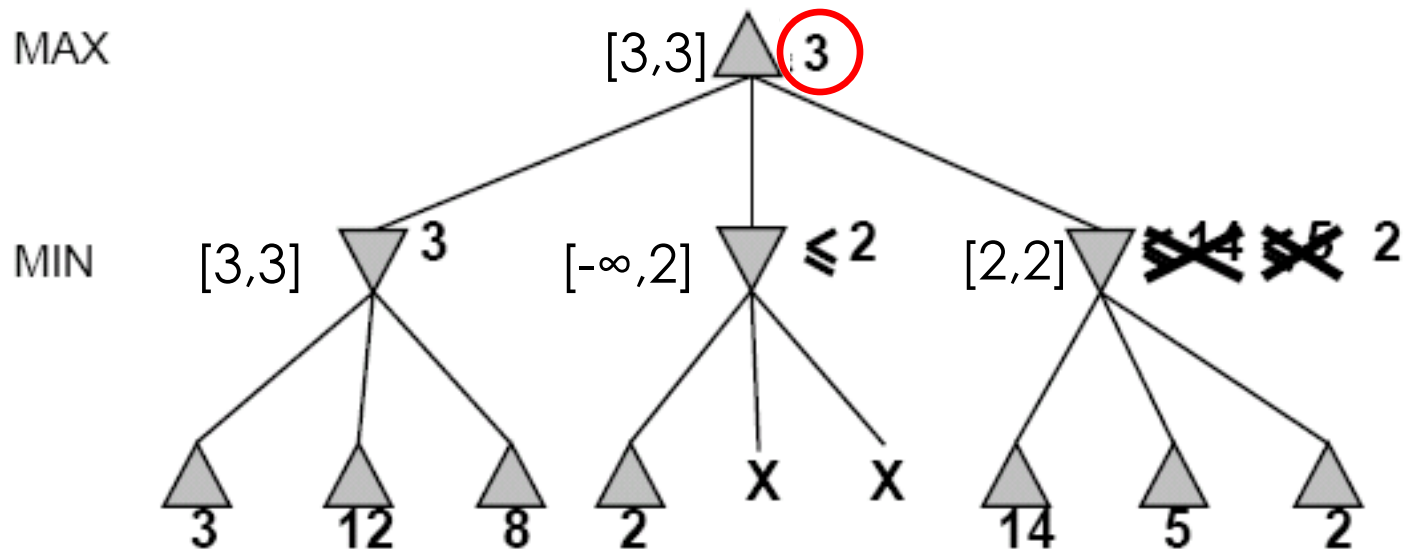
Ejemplo Alpha-Beta (continúa)



Ejemplo Alpha-Beta (continúa)



Ejemplo Alpha-Beta (continúa)



$$\begin{aligned}
 \text{MINIMAX (raíz)} &= \max (\min (3, 12, 8), \min (2, x, y), \min (14, 5, 2)) \\
 &= \max (3, \min (2, x, y), 2) \\
 &= \max (3, z, 2) \quad \text{donde } z = \min (2, x, y) \leq 2 \\
 &= 3
 \end{aligned}$$

Algoritmo Alpha-Beta

function ALPHA-BETA-SEARCH(*estado*) **returns** *una acción*

$v \leftarrow \text{MAX-VALOR}(\text{estado}, -\infty, +\infty)$

return la acción en $\text{ACCIONES}(\text{estado})$ con valor v

function MAX-VALOR(*estado*, α , β) **returns** un valor utilidad

if TEST-TERMINAL (*estado*) **then return** UTILIDAD(*estado*)

$v \leftarrow -\infty$

for each a in $\text{ACCIONES}(\text{estado})$ **do**

$v \leftarrow \text{MAX}(v, \text{MIN-VALOR}(\text{RESULTADO}(s, a), \alpha, \beta))$

if $v \geq \beta$ **then return** v

$\alpha \leftarrow \text{MAX}(\alpha, v)$

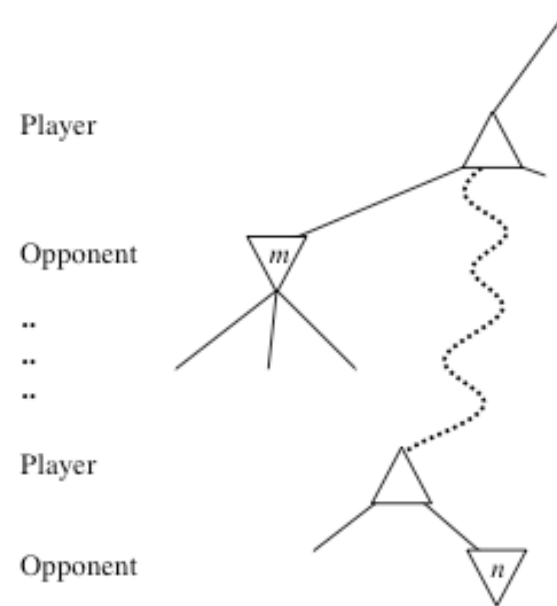
return v

Algoritmo

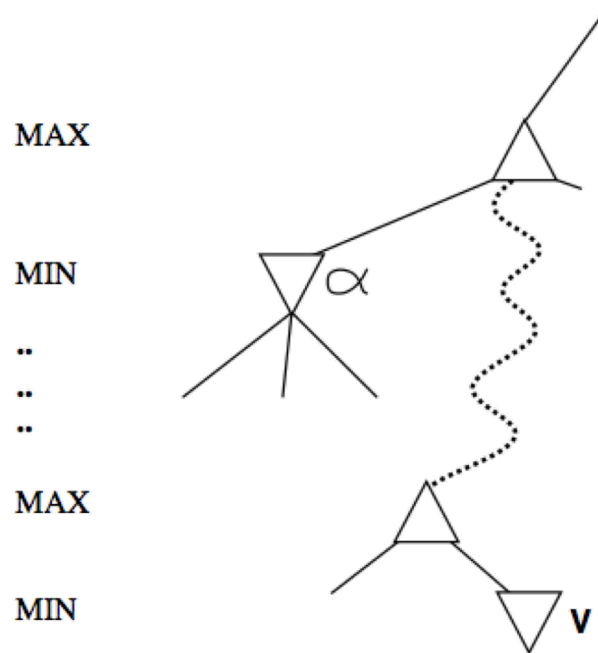
```
function MIN-VALOR(estado,  $\alpha$ ,  $\beta$ ) returns un valor utilidad  
  if TEST-TERMINAL(estado) then return UTILIDAD(estado)  
   $v \leftarrow +\infty$   
  for each a in ACCIONES(estado) do  
     $v \leftarrow \text{MIN}(v, \text{MAX-VALOR}(\text{RESULTADO}(s, a), \alpha, \beta))$   
    if  $v \leq \alpha$  then return  $v$   
     $\beta \leftarrow \text{MIN}(\beta, v)$   
  return  $v$ 
```

Caso general poda alfa-beta

- Considerar un nodo n en algún lugar del árbol al que el jugador tiene opción de mover.
- Si el jugador tiene una mejor opción en
 - El nodo padre de n
 - O en cualquier punto de elección superior/anterior
- n **nunca** será alcanzado en el juego
- Por lo tanto, cuando se tiene suficiente información sobre n , se puede podar.



Caso general poda alfa-beta



α es el mejor valor (para **max**) encontrado hasta el momento en el camino en curso

Si v es peor que α , max lo evitará $\Rightarrow$ poda esta rama

Define β de forma similar para **min**



Comentarios finales sobre la poda Alfa-Beta

- La poda no afecta los resultados finales
- Se puede podar subárboles completos.
- El orden de los movimientos puede mejorar la efectividad de la poda



Comentarios finales sobre la poda Alfa-Beta

- Con una “ordenación perfecta,” complejidad temporal sería $O(b^{m/2})$
 - Esto significa reducir el factor de ramificación efectivo de b a $\sqrt{b}$. Por ejemplo en ajedrez de 35 a 6.
 - La poda Alfa-beta puede ir al doble de profundidad que minimax en el mismo tiempo.
 - Los mejores movimientos se denominan **Killer moves**.
- Estados repetidos son posibles.
 - Idea: Almacenarlos en memoria (**transposition moves**, movimientos que dan lugar al mismo estado) para no reevaluar



Juegos con información imperfecta

- Minimax y la poda alfa-beta requieren evaluaciones de demasiados.
- Puede ser impracticable en una suma de tiempo razonable.
- SHANNON (1950):
 - Cortar la búsqueda antes (reemplazando **TEST-TERMINAL** por **TEST-CORTE**)
 - Aplicar una **función de evaluación heurística EVAL** (reemplazando la función de utilidad de alpha-beta)

Corte de la búsqueda

Cambio:

if TEST-TERMINAL (*estado*) **then return** UTILIDAD(*estado*)

por

- **if** TEST-CORTE(*estado*,*profundidad*) **then return** EVAL(*estado*)

- Introduce un límite de profundidad fijo

- Se selecciona de forma que la suma de tiempo no exceda lo que las reglas del juego permiten. Más robusto con profundización iterativa, devuelve el mov. Más profundo completado.

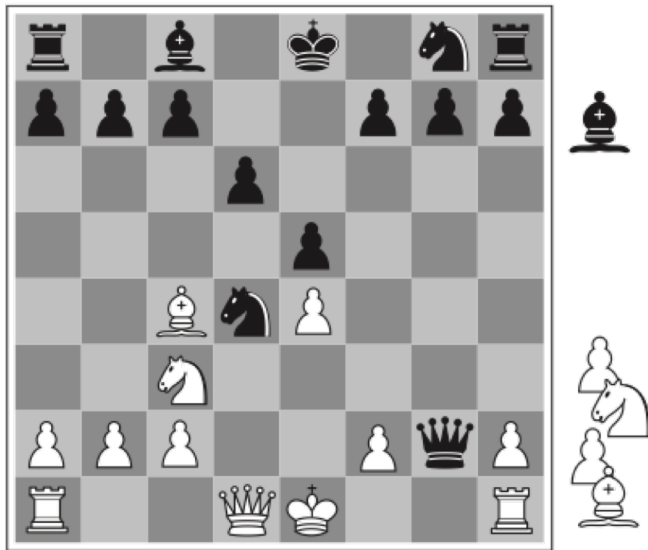
- Cuando el corte ocurre se realiza la evaluación

En ajedrez, uponer que tenemos 100 segundos, exploramos 10^4 nodos/segundo $\Rightarrow 10^6$ nodos por movimiento $\approx 35^{8/2}$, α - β alcanza profundidad 8 $\Rightarrow$ buen juego de ajedrez

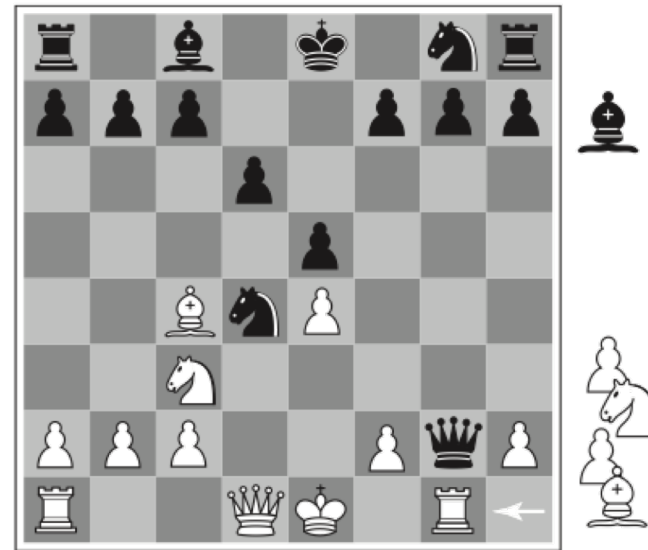
Función heurística EVAL

- Idea: producir una estimación de la utilidad esperada del juego para una posición dada.
- Las prestaciones dependen de la calidad de EVAL.
- Requisitos:
 - EVAL debería ordenar los nodos terminales de la misma forma que UTILITY.
 - El computo de la función no debe ser costoso.
 - Para estados no terminales, EVAL debería estar fuertemente correlacionada con las oportunidades reales de ganar.

Funciones de evaluación



Negras tienen ventaja de un caballo y dos peones, suficiente para ganar



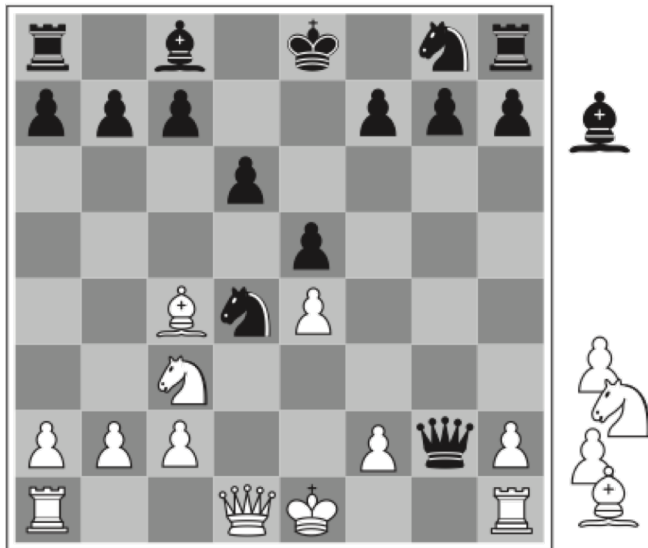
Blancas comen reina, dando ventaja suficiente para ganar

$$Eval(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s)$$

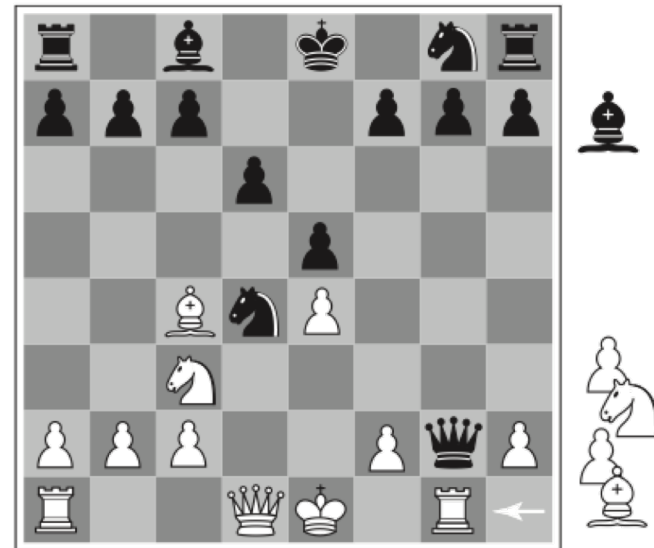
Función lineal del valor de las piezas

Suma asume Independencia => No es cierto, los caballos valen más cuando hay pocas piezas, porque tienen más libertad de movimientos

Funciones de evaluación



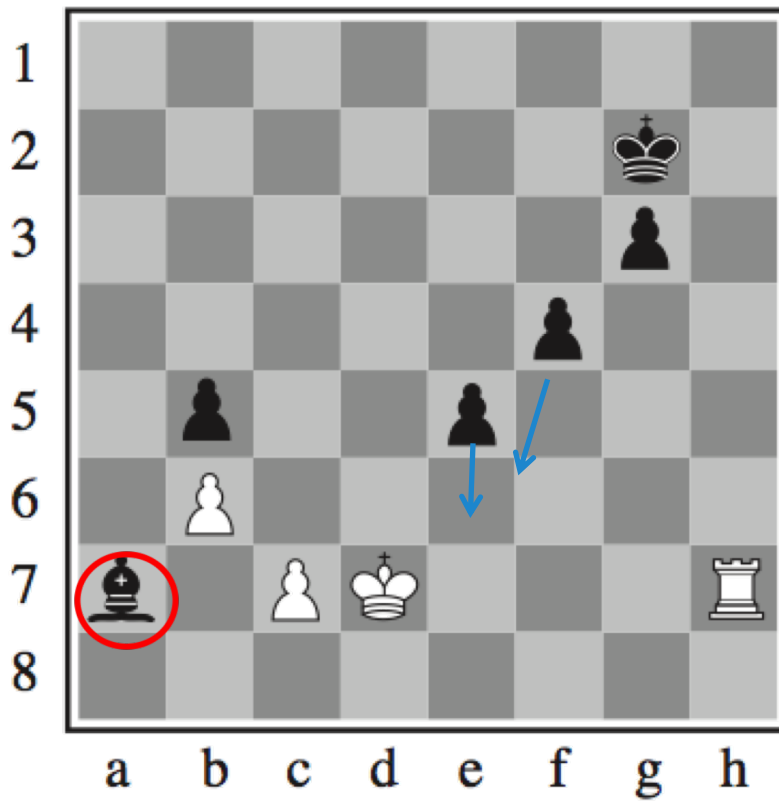
Negras tienen ventaja de un caballo y dos peones, suficiente para ganar



Blancas comen reina, dando ventaja suficiente para ganar

Este ejemplo muestra que un movimiento puede cambiar el valor en un paso. La evaluación debería ser sólo aplicada a posiciones estables (quiescent). Las posiciones no estables deben extenderse más allá hasta que se alcancen posiciones estables.

Efecto Horizonte

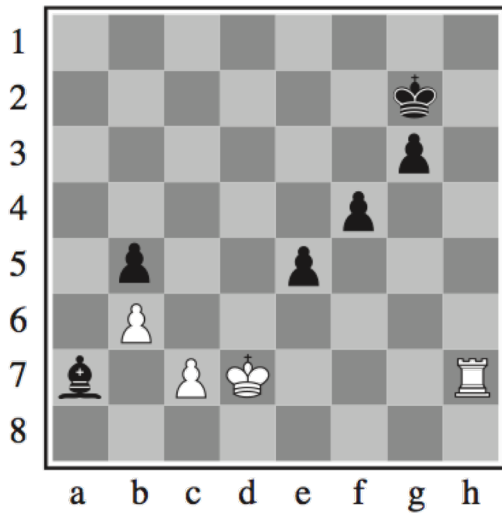


Una búsqueda de profundidad fija piensa que puede evitar Comer el alfil

Si el horizonte es cuatro, lo parece pero el sacrificio de los peones no evita lo inevitable

SOLUCION: Extensión singular, Extender sólo un poco más movimiento que se considera mejor.

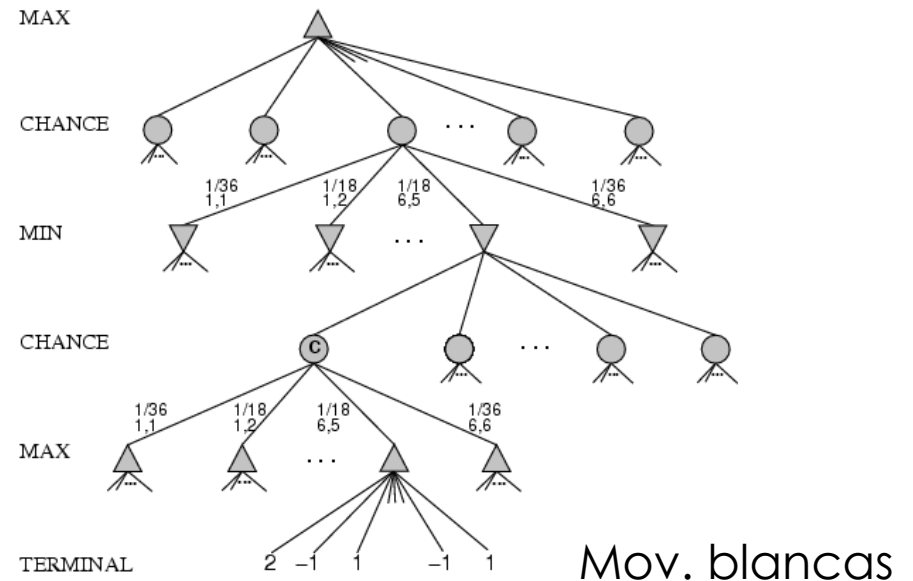
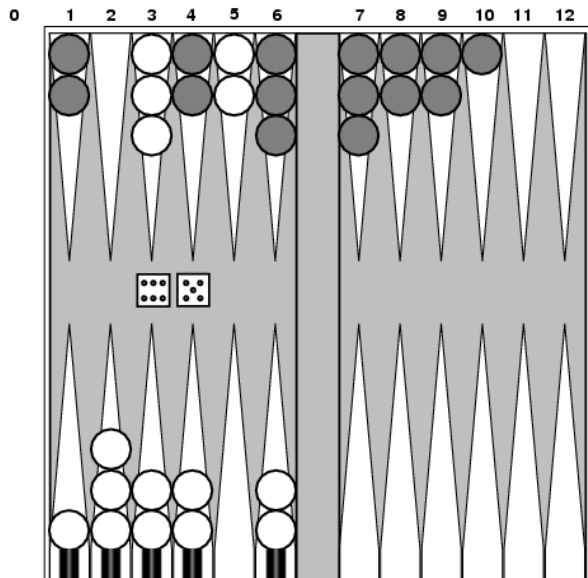
Otras mejoras



Forward Pruning: En cada turno, sólo se consideran n mejores movimientos

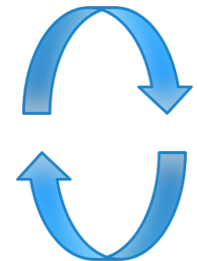
- Esta aproximación puede ignorar el mejor movimiento al podarlo.
- El uso de estadísticas obtenidas de experiencia previa mejoran el algoritmo.
- Utilización de aperturas y fines de juegos.

Juegos que incluyen azar



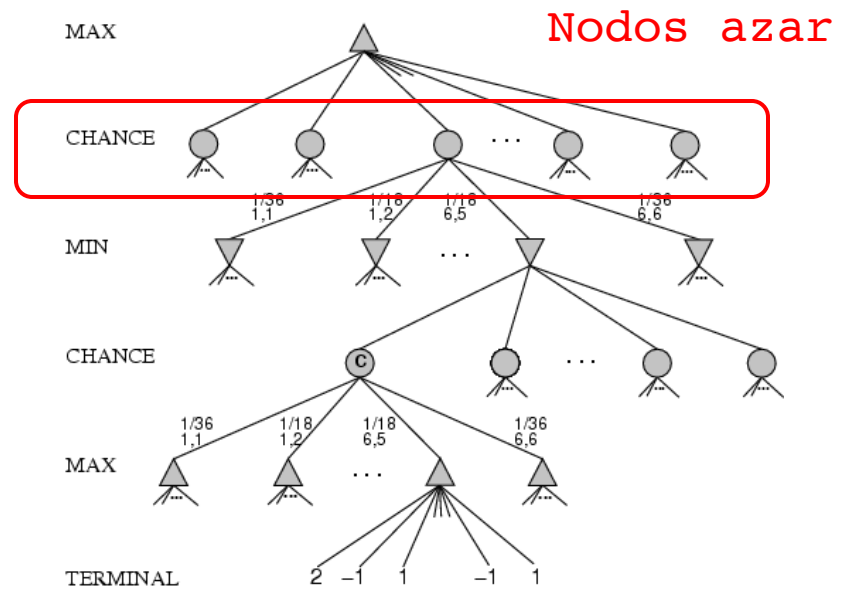
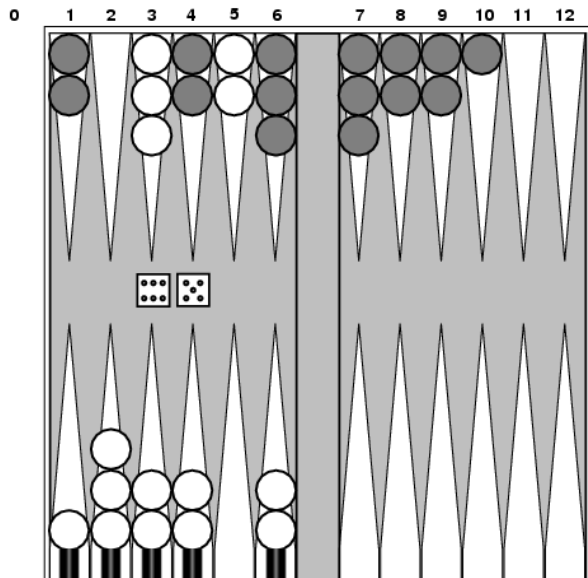
- Si hay una pieza oponente se come, si hay más no se puede mover
- Movimientos posibles (5-10,5-11), (5-11,19-24),(5-10,10-16) y(5-11,11-16)

Mov. blancas



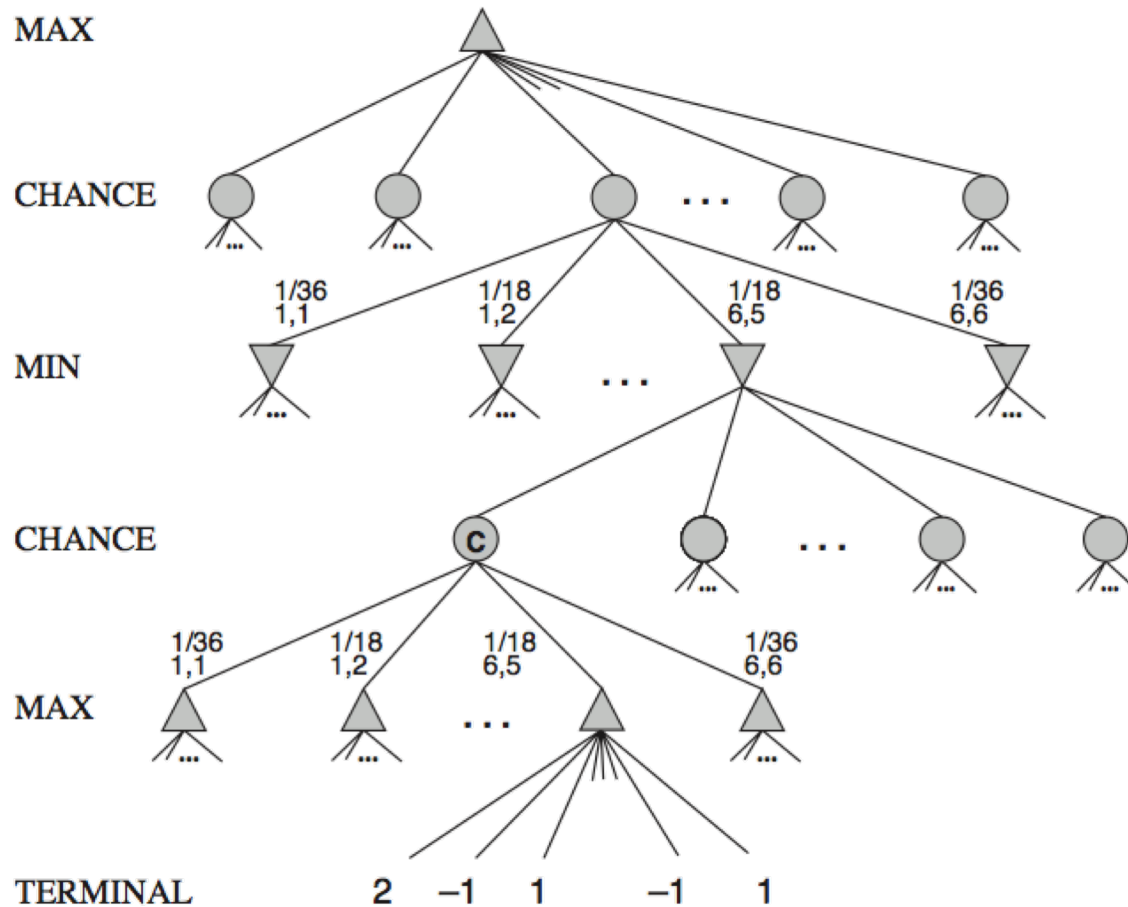
Mov. negras

Juegos que incluyen azar

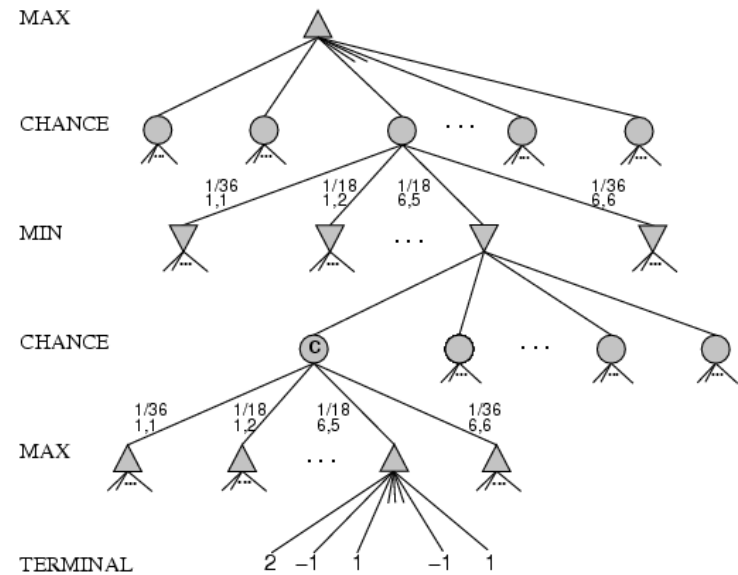
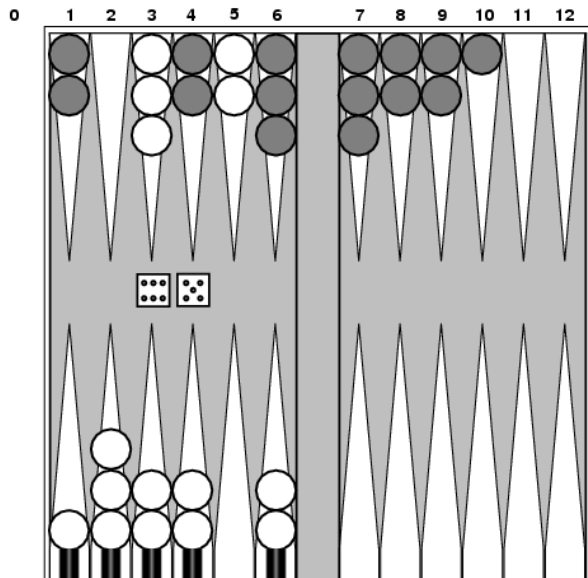


- Movimientos posibles (5-10,5-11), (5-11,19-24),(5-10,10-16) y(5-11,11-16)
- Los seis dobles [1,1].. [6,6] con probabilidad $1/36$, y los otros con $1/18$

Juegos que incluyen azar



Juegos que incluyen azar



- $[1,1]$, $[6,6]$ chance $1/36$, all other chance $1/18$
- No podemos calcular valores definidos de minimax, sólo **valores esperados**.

Valor Expected minimax

EXPECTED-MINIMAX (s)=

UTILITY(s)

If *TEST-TERMINAL*(s)

$\max_a$ EXPECTED-MINIMAX(*RESULT*(s, a))

If *JUGADOR*(s) es MAX

$\min_a$ EXPECTED-MINIMAX(*RESULT*(s, a))

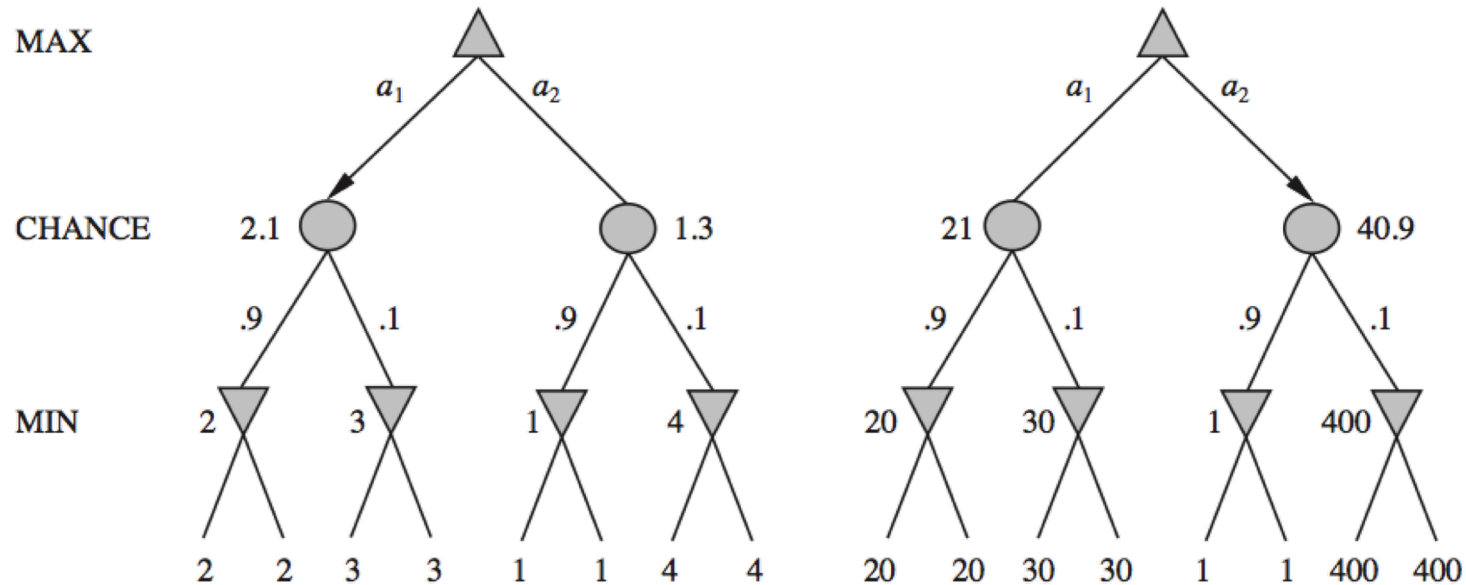
If *JUGADOR*(s) es MIN

$\sum_r P(r) \cdot$ EXPECTED-MINIMAX(*RESULT*(s, a))

If *JUGADOR*(s) es AZAR

donde r representa valores del dado (o probabilidad del evento)

Evaluación de posición con nodos azar



- IZQ, A1 gana
- Derecha A2 gana
- El programa se comporta de forma diferente si cambia la escala de la función de evaluación.
- El comportamiento se preserva con transformaciones lineales de la función de evaluación.



Resumen

- Los juegos ilustran aspectos importantes de la IA
 - La perfección es inalcanzable -> aproximación
 - La incertidumbre restringe la asignación de valores a estados.
 - Decisiones optimas dependen de la información del estado, no del estado real



Universidad
Zaragoza



Inteligencia Artificial

(30223) Grado en Ingeniería Informática

lección 5. Búsqueda con adversario 5.1 a 5.5 (AIMA), lectura del resto del capítulo.